



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis —
List + Tuple
10 marks

**Q2**

Customer Name Analysis —
String + List
10 marks

**Q3**

Product Inventory Analysis —
List + Tuple
10 marks

**Q4**

Text Analysis — String + List
10 marks

**Q5**

Employee Skill Analysis —
Tuple + List + String
10 marks

**Q6**

Transaction Analysis — Tuple +
List
10 marks

**Q7****Q1**

Student Performance Analysis — List + Tuple

10
marks

A college stores the following student records:

```
students = [  
    ("Arun", [78, 85, 92, 67]),  
    ("Bala", [45, 67, 55, 49]),  
    ("Charan", [90, 92, 88, 95]),  
    ("Divya", [65, 72, 70, 68]),  
    ("Esha", [88, 76, 91, 84])  
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.

Password Security Analysis —
String + List
10 marks

Q8 
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9 
Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10 
Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

9. Identify the top two students.

10. Explain which parts of the structure are mutable and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Divya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

Your Code

```

1 # Q1 Student Performance Analysis - List + T
2 a=[("Arun",[78,85,92,67]),
3 ("Bala",[45,67,55,49]),
4 ("Charan",[90,92,88,95]),
5 ("Divya",[65,72,70,68]),
6 ("Esha",[88,76,91,84])]
7 # 1. Calculate the total and average marks o
8 b=[]
9 c=[]
10 for d in a:
11     e=sum(d[1])
12     f=e/len(d[1])
13     b.append((d[0],e))
14     c.append((d[0],f))
15 print("Student Totals:")
16 for d in b:
17     print(d[0],":",d[1])
18 print()
19 print("Student Averages:")
20 for d in c:
21     print(d[0],":","{: .2f}".format(d[1]))
22 # 2. Identify the student with the highest to
23 g=max(b,key=lambda x:x[1])
24 # 3. Identify the student with the lowest av
25 h=min(c,key=lambda x:x[1])
26 print()
27 print("Highest Total:",g[0])
28 print("Lowest Average:",h[0])
29 print()
30 # 4. Find students whose average is greater
31 i=sum([x[1] for x in c])/len(c)
32 j=[x[0] for x in c if x[1]>i]
33 print("Above Class Average:")
34 print(", ".join(j))
35 print()
36 # 5. Find students who scored above 80 in at
37 k=[]
38 for d in a:
39     l=[x for x in d[1] if x>80]
40     if len(l)>=3:
41         k.append(d[0])
42 print("Students with 3 or more marks above 8
43 if k:
44     print(", ".join(k))
45 else:

```

```

46     print()
47 print()
48 # 6. Find the highest mark in each subject.
49 m=[0,0,0,0]
50 for d in a:
51     for n in range(4):
52         if d[1][n]>m[n]:
53             m[n]=d[1][n]
54 print("Subject-wise Highest:")
55 for n in range(4):
56     print("Subject",n+1,":",m[n])
57 print()
58 # 7. Identify students who scored below 50 i
59 o=[]
60 for d in a:
61     for p in d[1]:
62         if p<50:
63             o.append(d[0])
64             break
65 print("Students with a mark below 50:")
66 print(", ".join(o))
67 print()
68 # 8. Sort the students based on their averag
69 q=sorted(c,key=lambda x:x[1],reverse=True)
70
71
72
73 # 9. Identify the top two students.
74 print("Top 2:")
75 print(q[0][0])
76 print(q[1][0])
77
78
79 # 10. Explain which parts of the structure a
80 # In the list "a", the outer structure is a
81 # Each student record is a tuple, which is i
82 # Inside each tuple, the marks are stored in

```



Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING

Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

Q2

Customer Name Analysis —
String + List
10 marks

Q3

Product Inventory Analysis —
List + Tuple
10 marks

Q4

Text Analysis — String + List
10 marks

Q5

Employee Skill Analysis —
Tuple + List + String
10 marks

Q6

Transaction Analysis — Tuple +
List
10 marks

Q7

Q2 Customer Name Analysis — String + List 10 marks

An e-commerce company receives customer names in inconsistent formats:

```
names = [  
    " Arun Kumar ",  
    "ARUN KUMAR",  
    " arun kumar",  
    "Bala Krishnan",  
    "BALA KRISHNAN ",  
    "Charan",  
    " charan ",  
    "DIVYA DEVI",  
    "Divya Devi"  
]
```

Analyze the customer data.

Tasks

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.
6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

8. Count the number of characters in each normalized name.

9. Sort the unique names alphabetically.

10. Determine which customer name occurs most frequently.

Expected Output:

Normalized:

['arun kumar', 'arun kumar', 'arun kumar',
'bala krishnan', 'bala krishnan',
'charan', 'charan',
'divya devi', 'divya devi']

Unique Customers:

['arun kumar', 'bala krishnan', 'charan', 'divya devi']

Number of Unique Customers: 4

Duplicate Customers:

arun kumar
bala krishnan
charan
divya devi

Names with Multiple Words:

arun kumar
bala krishnan
divya devi

Longest Name:

bala krishnan

Starts with A or D:

arun kumar
divya devi

Your Code

```
1 a=[
2 " Arun Kumar ",
3 "ARUN KUMAR",
4 " arun kumar",
5 "Bala Krishnan",
```

```
6 "BALA KRISHNAN ",
7 "Charan",
8 " charan ",
9 "DIVYA DEVI",
10 "Divya Devi"
11 ]
12
13 b=[]
14 for c in a:
15     b.append(c.strip().lower())
16 # 1. Remove unnecessary leading and trailing
17 # 2. Convert all names to lowercase.
18 print("Normalized:")
19 print(b)
20 # 3. Identify duplicate customers.
21 d=[]
22 for c in b:
23     if c not in d:
24         d.append(c)
25 e=[]
26 for c in d:
27     if b.count(c)>1:
28         e.append(c)
29 print()
30 print("Duplicate Customers:")
31 for c in e:
32     print(c)
33 # 4. Count the number of unique customers.
34 print()
35 print("Number of Unique Customers:",len(d))
36
37 # 5. Find customers whose names contain more
38 print()
39 print("Names with Multiple Words:")
40 for c in d:
41     if " " in c:
42         print(c)
43 # 6. Find the longest customer name.
44 f=d[0]
45 for c in d:
46     if len(c)>len(f):
47         f=c
48 print()
49 print("Longest Name:")
50 print(f)
```

```
51 # 7. Find customers whose names start with '
52 print()
53 print("Starts with A or D:")
54 for c in d:
55     if c[0]=="a" or c[0]=="d":
56         print(c)
57 # 8. Count the number of characters in each
58 print()
59 print("Character Count in Each Name:")
60 for c in d:
61     print(c,len(c))
62 # 9. Sort the unique names alphabetically.
63 g=sorted(d)
64 print()
65 print("Sorted Unique Names:")
66 print(g)
67
68
69
70 # 10. Determine which customer name occurs m
71 h=d[0]
72 i=0
73 for c in d:
74     if b.count(c)>i:
75         i=b.count(c)
76         h=c
77 print()
78 print("Most Frequent Customer:")
79 print(h)
```

Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC



CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis —
List + Tuple
10 marks

Q2

Customer Name Analysis —
String + List
10 marks

Q3

Product Inventory Analysis —
List + Tuple
10 marks

Q4

Text Analysis — String + List
10 marks

Q5

Employee Skill Analysis —
Tuple + List + String
10 marks

Q6

Transaction Analysis — Tuple +
List
10 marks

Q7**Q3**

Product Inventory Analysis — List + Tuple

10
marks

A supermarket stores product information as:

```
products = [  
    ("Laptop", 5, 55000),  
    ("Mouse", 25, 700),  
    ("Keyboard", 8, 1200),  
    ("Monitor", 12, 15000),  
    ("Printer", 4, 12000),  
    ("Headset", 15, 2500)  
]
```

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory.

Tasks

1. Calculate the inventory value of every product.
2. Calculate the total inventory value.
3. Identify products whose quantity is less than 10.
4. Identify products whose price is greater than ₹10,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.

Password Security Analysis —
String + List
10 marks

Q8 
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9 
Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10 
Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

10. Identify products contributing more than ₹50,000 to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 37500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
1 .
2 p=[ ("Laptop",5,55000) ,
3  ("Mouse",25,700) ,
4  ("Keyboard",8,1200) ,
5  ("Monitor",12,15000) ,
6  ("Printer",4,12000) ,
7  ("Headset",15,2500) ]
8 #1
9 v=[]
10 for a in p:
```



```
11  b=a[1]*a[2]
12  v.append(b)
13  print("Inventory Values:")
14  for i in range(len(p)):
15      print(p[i][0],":",v[i])
16  print()
17  #2
18  print("Total Inventory Value:",sum(v))
19  print()
20  #3
21  print("Low Stock:")
22  for a in p:
23      if a[1]<10:
24          print(a[0])
25  print()
26
27  #4
28  print("Price > 10000:")
29  for a in p:
30      if a[2]>10000:
31          print(a[0])
32  print()
33
34  #5
35  m=p[0]
36  for a in p:
37      if a[2]>m[2]:
38          m=a
39  print("Most Expensive:")
40  print(m[0])
41  print()
42
43  #6
44  n=p[0]
45  for i in range(len(p)):
46      if v[i]>n[1]*n[2]:
47          n=p[i]
48  print("Highest Inventory Value:")
49  print(n[0])
50  print()
51
52  #7
53  q=sorted(p,key=lambda x:x[2])
54  print("Sorted by Price:")
55  for a in q:
```

```
56 print(a[0],a[1],a[2])
57 print()
58
59
60
61
62 #8
63 r=sorted(p,key=lambda x:x[1])
64 print("Sorted by Quantity:")
65 for a in r:
66     print(a[0],a[1],a[2])
67 print()
68
69 #9
70 s=0
71 for a in p:
72     s+=a[1]
73 print("Total Quantity:",s)
74 print()
75
76
77 #10
78 print("Inventory Value > 50000:")
79 for i in range(len(p)):
80     if v[i]>50000:
81         print(p[i][0])
```

Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING

Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks



Q2

Customer Name Analysis —
String + List
10 marks



Q3

Product Inventory Analysis —
List + Tuple
10 marks



Q4

Text Analysis — String + List
10 marks



Q5

Employee Skill Analysis —
Tuple + List + String
10 marks



Q6

Transaction Analysis — Tuple +
List
10 marks



Q7



Q4 Text Analysis — String + List

10 marks

A university receives the following feedback:

```
feedback = ""
```

Python programming is easy to learn.

Python programming is powerful.

Learning Python requires practice.

Practice makes programming skills better.

```
"""
```

Analyze the text.

Tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Total Words: 21

Unique Words: 16

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

Your Code

```

1 feedback = """
2 Python programming is easy to learn.
3 Python programming is powerful.
4 Learning Python requires practice.
5 Practice makes programming skills better.
6 """
7 a = feedback.lower()
8 print("#1 Lowercase Text:")
9 print(a)
10 #2 Remove punctuation.
11 b = ""
12 for c in a:
13     if c.isalpha() or c.isspace():
14         b += c
15 print("\n#2 Text without punctuation:")
16 print(b)
17 #3 Convert the text into a list of words.
18 d = b.split()
19 print("\n#3 List of words:")

```

```
20 print(d)
21 #4 Count the total number of words.
22 print("\n#4 Total number of words:")
23 print(len(d))
24 #5 Find the number of unique words.
25 e = list(set(d))
26 print("\n#5 Number of unique words:")
27 print(len(e))
28 #6 Find the frequency of "python".
29 print("\n#6 Frequency of \"python\":")
30 print(d.count("python"))
31 #7 Find the frequency of "programming".
32 print("\n#7 Frequency of \"programming\":")
33 print(d.count("programming"))
34 #8 Identify words appearing more than once.
35 print("\n#8 Words appearing more than once:")
36 f = []
37 for g in e:
38     if d.count(g) > 1:
39         f.append(g)
40 for g in sorted(f):
41     print(g)
42 #9 Find the longest word.
43 print("\n#9 Longest word:")
44 h = ""
45 for g in e:
46     if len(g) > len(h):
47         h = g
48 print(h)
49 #10 Sort the unique words alphabetically and
50 print("\n#10 Unique words sorted alphabetica
51 for g in sorted(e):
52     print(g)
53 print("\nPercentage of unique words:")
54 i = (len(e) / len(d)) * 100
55 print(i)
```



Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC



CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

**Q2**

Customer Name Analysis —
String + List
10 marks

**Q3**

Product Inventory Analysis —
List + Tuple
10 marks

**Q4**

Text Analysis — String + List
10 marks

**Q5**

Employee Skill Analysis —
Tuple + List + String
10 marks

**Q6**

Transaction Analysis — Tuple +
List
10 marks

**Q7****Q5**

Employee Skill Analysis — Tuple + List + String

10
marks

An organization stores:

```
employees = [  
    ("E101", "Arun", ["Python", "SQL", "ML"]),  
    ("E102", "Bala", ["Java", "SQL", "HTML"]),  
    ("E103", "Charan", ["Python", "ML", "SQL"]),  
    ("E104", "Divya", ["Python", "Java", "Cloud"]),  
    ("E105", "Esha", ["Python", "ML", "Cloud"])  
]
```

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

Your Code

```

1 # employees data
2 a=[
3 ("E101","Arun",["Python","SQL","ML"]),
4 ("E102","Bala",["Java","SQL","HTML"]),
5 ("E103","Charan",["Python","ML","SQL"]),
6 ("E104","Divya",["Python","Java","Cloud"]),
7 ("E105","Esha",["Python","ML","Cloud"])
8 ]
9
10 # 1. Find all employees who know Python.
11 b=[x[1] for x in a if "Python" in x[2]]
12 print("Python:")
13 print(", ".join(b))
14 print()
15
16 # 2. Find employees who know both Python and
17 c=[x[1] for x in a if "Python" in x[2] and "
18 print("Python + ML:")
19 print(", ".join(c))

```

```
20 print()
21
22 # 3. Find employees who know SQL but not Jav
23 d=[x[1] for x in a if "SQL" in x[2] and "Jav
24 print("SQL but not Java:")
25 print(", ".join(d))
26 print()
27
28 # 4. Find employees who know either Java or
29 e=[x[1] for x in a if "Java" in x[2] or "Clo
30 print("Java or Cloud:")
31 print(", ".join(e))
32 print()
33
34 # 5. Count employees having each skill.
35 f={}
36 for x in a:
37     for y in x[2]:
38         if y in f:
39             f[y]+=1
40         else:
41             f[y]=1
42
43
44
45 # 6. Identify the most common skill.
46 g=max(f,key=f.get)
47 print("Most Common Skill:")
48 print(g)
49 print()
50
51
52
53 # 7. Identify employees having at least thre
54 h=[x[1] for x in a if len(x[2])>=3]
55 print("Employees with 3 skills:")
56 print(", ".join(h))
57 print()
58
59 # 8. Find employees who do not know Python.
60 i=[x[1] for x in a if "Python" not in x[2]]
61 print("Without Python:")
62 print(", ".join(i))
63 print()
64
```

```
65 # 9. Sort employes based on the number of sk
66 j=sorted(a,key=lambda x:len(x[2]),reverse=Tr
67 # (theory: sorted list j has employees from
68
69 # 10. Determine which employees are sitable
70 k=[x[1] for x in a if "Python" in x[2] and "
71 print("Suitable for Python + ML:")
72 print(", ".join(k))
```

Input

stdin input

Output



Run



Save



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis — List + Tuple
10 marks

**Q2**

Customer Name Analysis — String + List
10 marks

**Q3**

Product Inventory Analysis — List + Tuple
10 marks

**Q4**

Text Analysis — String + List
10 marks

**Q5**

Employee Skill Analysis — Tuple + List + String
10 marks

**Q6**

Transaction Analysis — Tuple + List
10 marks

**Q7**

Q6 Transaction Analysis — Tuple + List

10 marks

A bank stores transactions:

```
transactions = [  
    ("T101", "Deposit", 5000),  
    ("T102", "Withdraw", 2000),  
    ("T103", "Deposit", 8000),  
    ("T104", "Withdraw", 1500),  
    ("T105", "Deposit", 3000),  
    ("T106", "Withdraw", 7000)  
]
```

Analyze the transaction history.

Tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than ₹4,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 t=[("T101", "Deposit", 5000),
2   ("T102", "Withdraw", 2000),
3   ("T103", "Deposit", 8000),
4   ("T104", "Withdraw", 1500),
5   ("T105", "Deposit", 3000),
6   ("T106", "Withdraw", 7000)]
7 # 1. Calculate total deposits.
8 a=0
9 for i in t:
10     if i[1]=="Deposit":
11         a+=i[2]
12 # 2. Calculate total withdrawals.
13 b=0
14 for i in t:
15     if i[1]=="Withdraw":
16         b+=i[2]
17 # 3. Calculate the net balance.
18 c=a-b
19 # 4. Find the largest deposit.
20 d=0
21 for i in t:
22     if i[1]=="Deposit" and i[2]>d:
```

```
23         d=i[2]
24 # 5. Find the largest withdrawal.
25 e=0
26 for i in t:
27     if i[1]=="Withdraw" and i[2]>e:
28         e=i[2]
29 # 6. Identify all transactions greater than
30 f=[]
31 for i in t:
32     if i[2]>4000:
33         f.append(i[0])
34 # 7. Count deposits and withdrawals.
35 g=0
36 h=0
37 for i in t:
38     if i[1]=="Deposit":
39         g+=1
40     else:
41         h+=1
42 # 8. Find the average transaction amount.
43 s=0
44 for i in t:
45     s+=i[2]
46 j=s/len(t)
47 # 9. Sort transactions based on amount.
48 k=sorted(t,key=lambda x:x[2])
49 # 10. Identify wheter total withdrawals exce
50 l="No"
51 if b>0.5*a:
52     l="Yes"
53 print("Total Deposit:",a)
54 print("Total Withdrawal:",b)
55 print("Net Balance:",c)
56 print()
57 print("Largest Deposit:",d)
58 print("Largest Withdrawal:",e)
59 print()
60 print("Transactions > 4000:")
61 for i in f:
62     print(i)
63 print()
64 print("Deposits:",g)
65 print("Withdrawals:",h)
66 print()
67 print("Average Transaction:",round(j,2))
```

```
68 print()  
69 print("Withdrawals > 50% of Deposits:")  
70 print(1)
```

Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC



CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis

— List + Tuple

10 marks

Q2

Customer Name Analysis —

String + List

10 marks

Q3

Product Inventory Analysis —

List + Tuple

10 marks

Q4

Text Analysis — String + List

10 marks

Q5

Employee Skill Analysis —

Tuple + List + String

10 marks

Q6

Transaction Analysis — Tuple +

List

10 marks

Q7**Q7**

Password Security Analysis — String + List

10

marks

A system receives:

```
passwords = [  
    "Python@123",  
    "python123",  
    "PYTHON@123",  
    "Py@45",  
    "Python123",  
    "Java#4567",  
    "Admin@2026"  
]
```

A valid password must have:

- At least 8 characters
- Uppercase
- Lowercase
- Digit
- Special character

Analyze:

1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of case.
8. Sort passwords based on length.
9. Determine the percentage of valid passwords.

Expected Output:

Valid:

Python@123

Java#4567

Admin@2026

Invalid:

python123 → No uppercase, No special character

PYTHON@123 → No lowercase

Py@45 → Less than 8 characters

Python123 → No special character

Valid Count: 3

Invalid Count: 4

Longest Password:

Python@123

Python-related passwords:

Python@123

python123

PYTHON@123

Python123

Valid Percentage: 42.86%

Your Code

```
1 def q1():
2     a=[(1, 85), (2, 90), (3, 78)]
3     for i, g in a:
4         print(i, g)
5 q1()
6
7 # Q2: Customer Name Analysis
8 def q2():
```

```
9     a=["John Doe", "Jane Smith"]
10     for n in a:
11         print(n.upper())
12 q2()
13
14 # Q3: Product Inventory Analysis
15 def q3():
16     a=[("Apple", 10), ("Banana", 5)]
17     print(a)
18 q3()
19
20 # Q4: Text Analysis
21 def q4():
22     a="hello world"
23     print(a.split())
24 q4()
25
26 # Q5: Employee Skill Analysis
27 def q5():
28     a=("Dev", ["Python", "SQL"])
29     print(a)
30 q5()
31
32 # Q6: Transaction Analysis
33 def q6():
34     a=[(100, "USD"), (200, "EUR")]
35     print(a)
36 q6()
37
38 # Q7: Password Security Analysis
39 p=["Python@123", "python123", "PYTHON@123"]
40 v=[]
41 iv=[]
42 d={}
43 for a in p:
44     x=len(a)>=8
45     y=any(b.isupper() for b in a)
46     z=any(b.islower() for b in a)
47     u=any(b.isdigit() for b in a)
48     s=any(not b.isalnum() for b in a)
49     if x and y and z and u and s:
50         v.append(a)
51     else:
52         c=[]
53         if not x: c.append("Less than 8")
```

```

54         if not y: c.append("No uppercase")
55         if not z: c.append("No lowercase")
56         if not u: c.append("No digit")
57         if not s: c.append("No special c
58         iv.append(a)
59         d[a]=c
60     print("Valid:")
61     for a in v: print(a)
62     print("\nInvalid:")
63     for a in iv: print(a,"→",",", ".join(d[a])
64     print("\nValid Count:",len(v))
65     print("Invalid Count:",len(iv))
66     print("\nLongest Password:")
67     print(max(p,key=len))
68     print("\nPython-related passwords:")
69     for a in p:
70         if "python" in a.lower(): print(a)
71     print("\nSorted by length:")
72     for a in sorted(p,key=len): print(a)
73     print("\nValid Percentage:", "{:.2f}%".fo
74 q7()
75
76 # Q8: Code Analysis and Debugging
77 def q8():
78     a=[(1, "error"), (2, "ok")]
79     print(a)
80 q8()
81
82 # Q9: Word Frequency and Data Analysis
83 def q9():
84     a="apple banana apple".split()
85     print(a)
86 q9()
87
88 # Q10: Integrated Student Data Analysis
89 def q10():
90     a=["Student1", (90, 80)]
91     print(a)
92 q10()

```



Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

**Q2**

Customer Name Analysis —
String + List
10 marks

**Q3**

Product Inventory Analysis —
List + Tuple
10 marks

**Q4**

Text Analysis — String + List
10 marks

**Q5**

Employee Skill Analysis —
Tuple + List + String
10 marks

**Q6**

Transaction Analysis — Tuple +
List
10 marks

**Q7****Q8**

Code Analysis and Debugging — List + Tuple + String

10
marks

Consider the following program:

```
records = [  
    ("Arun ", "Python"),  
    ("ARUN", "Python"),  
    ("Bala ", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]  
  
names = []
```

```
for record in records:  
    name = record[0].strip().lower()
```

```
    if name not in names:  
        names.append(name)
```

```
    print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

7. What is the final output?

8. Modify the logic to find unique employees and their skills.

9. Analyze what happens if record[0] is changed instead.

10. Explain why modifying record[0] directly is not allowed.

Expected Output:

['arun', 'bala', 'charan']

Your Code

```

1 # 1. What happens during each iteration?
2 # Each iteration takes a tuple, cleans the n
3
4 # 2. What value is stored in record?
5 # A tuple containing a name and a skill.
6
7 # 3. What does record[0] represent?
8 # The name of the person.
9
10 # 4. Why are strip() and lower() used togeth
11 # To make sure " Arun " and "ARUN" are treat
12
13 # 5. Why does "Arun" appear only once?
14 # Because the code checks if the name is alr
15
16 # 6. Why does "Bala" appear only once?
17 # Same as Arun, the duplicate is blocked by
18
19 # 7. What is the final output?
20 # ['arun', 'bala', 'charan']
21
22 # 8. Modify the logic to find unique employe
23 r = [
24     (" Arun ", "Python"),
25     ("ARUN", "Python"),
26     (" Bala ", "Java"),
27     ("bala", "Java"),
28     ("Charan", "Python")
29 ]
30 d = {}
31 for a in r:
32     n = a[0].strip().lower()
```

```
33     s = a[1]
34     if n not in d:
35         d[n] = s
36 print(d)
37
38 # 9. Analyze what happens if record[0] is ch
39 # If we try to change record[0] directly, th
40
41 # 10. Explain why modifying record[0] direct
42 # Because tuples are immutable, meaning thei
```



Input

stdin input

Output



Run



Save



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis —
List + Tuple
10 marks

**Q2**

Customer Name Analysis —
String + List
10 marks

**Q3**

Product Inventory Analysis —
List + Tuple
10 marks

**Q4**

Text Analysis — String + List
10 marks

**Q5**

Employee Skill Analysis —
Tuple + List + String
10 marks

**Q6**

Transaction Analysis — Tuple +
List
10 marks

**Q7****Q9**

Word Frequency and Data Analysis — String + List + Tuple

10
marks

Consider:

text = "data science uses data analysis and data
visualization"

Analyze the string using Python.

Tasks

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:
(word, frequency)
8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Password Security Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

Frequency:

data → 3

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most Frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```

1 text = "data science uses data analysis and
2 a = text.split()                                # task
3 b = len(a)                                      # tas
4 c = list(set(a))                                # task
5 d = {}                                          # tas
6 for e in a:                                    # tas
7     if e in d:
8         d[e] = d[e] + 1
9     else:
10        d[e] = 1
11 f = None                                      # tas
12 g = -1                                       # tas
13 for k in d:
14     if d[k] > g:
15         g = d[k]
16         f = k
17 h = []                                       # tas
18 for k in d:
19     h.append((k, d[k]))
20 h.sort(key=lambda x: x[1], reverse=True)    #
21 i = []                                       # tas
22 for k in d:
23     if d[k] > 1:
24         i.append(k)

```

```
25 j = "" # tas
26 for k in a:
27     if len(k) > len(j):
28         j = k
29 k = [] # tas
30 for w in d:
31     if w.startswith('d'):
32         k.append(w)
33 l = (len(i) / b) * 100 # tas
34 print("Total Words:", b)
35 print("\nUnique Words:")
36 print(c)
37 print("\nFrequency:")
38 for x in d:
39     print(x, "→", d[x])
40 print("\nMost Frequent:")
41 print(f)
42 print("\nRepeated Words:")
43 print(i)
44 print("\nLongest Word:")
45 print(j)
46 print("\nWords beginning with 'd':")
47 print(k)
48 print("\nPercentage of repeated words:", l)
49 print("\nTuples (word, frequency) sorted by")
50 print(h)
```



Input

stdin input

Output



Run



Save

© 2026 **SIMATS Online Lab**. All rights reserved.



SIMATS Online Programming Lab

PYTHON PROGRAMMING



Jayadeep J
192511304RC

CO4_AT2_CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis —
List + Tuple
10 marks

**Q2**

Customer Name Analysis —
String + List
10 marks

**Q3**

Product Inventory Analysis —
List + Tuple
10 marks

**Q4**

Text Analysis — String + List
10 marks

**Q5**

Employee Skill Analysis —
Tuple + List + String
10 marks

**Q6**

Transaction Analysis — Tuple +
List
10 marks

**Q7****Q10**

Integrated Student Data Analysis — String + List + Tuple

10
marks

A college maintains:

```
students = [  
(" A001 ", "ARUN KUMAR", [78, 85, 92]),  
("A002", "Bala", [45, 67, 52]),  
(" a003", "arun kumar", [88, 91, 95]),  
("A004", "CHARAN", [72, 76, 81]),  
("A005", "Divya ", [55, 48, 62])  
]
```

Each record contains:

(Student ID, Student Name, Marks)

Analyze the complete dataset:

1. Normalize all student IDs using strip() and upper().
2. Normalize all student names using strip() and lower().
3. Calculate total and average marks.
4. Identify students who passed all subjects.
5. Identify students who failed at least one subject.
6. Find the highest total.
7. Find the lowest total.
8. Identify students with average above 75.
9. Detect duplicate student names after normalization.
10. Sort students based on average marks.
11. Find the top two students.

Password Security Analysis —
String + List
10 marks

Q8 
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9 
Word Frequency and Data
Analysis — String + List + Tuple
10 marks

Q10 
Integrated Student Data
Analysis — String + List + Tuple
10 marks

10/10 answered

12. Find the subject-wise highest marks.
13. Find the subject-wise average marks.
14. Count the number of students who scored above 80 in at least two subjects.
15. Identify the student who has the highest mark in Subject 3.
16. Explain which data structure is appropriate for student records, marks, and individual student identity, and justify your answer.

Expected Output:

Normalized Records:

A001 arun kumar [78, 85, 92]

A002 bala [45, 67, 52]

A003 arun kumar [88, 91, 95]

A004 charan [72, 76, 81]

A005 divya [55, 48, 62]

Totals:

arun kumar → 255

bala → 164

arun kumar → 274

charan → 229

divya → 165

Averages:

arun kumar → 85.00

bala → 54.67

arun kumar → 91.33

charan → 76.33

divya → 55.00

Passed All Subjects:

arun kumar

arun kumar

charan

Failed At Least One:

bala

divya

Highest Total:

arun kumar → 274

Lowest Total:

bala → 164

Average > 75:

arun kumar

arun kumar

charan

Duplicate Name:

arun kumar

Top 2:

arun kumar → 91.33

arun kumar → 85.00

Subject-wise Highest:

Subject 1 → 88

Subject 2 → 91

Subject 3 → 95

Subject-wise Average:

Subject 1 → 67.60

Subject 2 → 73.40

Subject 3 → 78.40

Your Code

```

1 students=[ (" A001 ", "ARUN KUMAR", [78, 85, 92])
2 ("A002", "Bala", [45, 67, 52]),
3 (" a003", "arun kumar", [88, 91, 95]),
4 ("A004", "CHARAN", [72, 76, 81]),
5 ("A005", "Divya ", [55, 48, 62])]
6
7 # 1-2 norm ids/names
8 x=[]
9 for a,b,c in students:
10     d=a.strip().upper()
11     e=b.strip().lower()
12     x.append((d,e,c))
13

```

```
14 print("Normalized Records:")
15 for a,b,c in x:
16     print(a,b,c)
17
18 # 3 totals/avg
19 print("Totals:")
20 for a,b,c in x:
21     print(b,"→",sum(c))
22
23 print("Averages:")
24 for a,b,c in x:
25     print(b,"→",format(sum(c)/len(c),'.2f'))
26
27 # 4 passed all / 5 failed any
28 print("Passed All Subjects:")
29 for a,b,c in x:
30     if all(y>=35 for y in c):
31         print(b)
32
33 print("Failed At Least One:")
34 for a,b,c in x:
35     if not all(y>=35 for y in c):
36         print(b)
37
38 # 6-7 highest/lowest total
39 m1=None
40 m2=None
41 for a,b,c in x:
42     k=sum(c)
43     if m1 is None or k>m1[1]:
44         m1=(b,k)
45     if m2 is None or k<m2[1]:
46         m2=(b,k)
47
48 print("Highest Total:")
49 print(m1[0],"→",m1[1])
50
51 print("Lowest Total:")
52 print(m2[0],"→",m2[1])
53
54 # 8 avg > 75
55 print("Average > 75:")
56 for a,b,c in x:
57     if sum(c)/len(c)>75:
58         print(b)
```

```
59
60 # 9 duplicate names
61 cnt={}
62 for a,b,c in x:
63     cnt[b]=cnt.get(b,0)+1
64 dup=[k for k,v in cnt.items() if v>1]
65
66 print("Duplicate Name:")
67 for a in dup:
68     print(a)
69
70 # 10 sort by avg
71 v=[(b,sum(c)/len(c)) for a,b,c in x]
72 v.sort(key=lambda p:p[1],reverse=True)
73
74 # 11 top 2
75 print("Top 2:")
76 for a,b in v[:2]:
77     print(a,"->",format(b, '.2f'))
78
79 # 12 subject-wise highest
80 h=[0,0,0]
81 for a,b,c in x:
82     for i in range(3):
83         if c[i]>h[i]:
84             h[i]=c[i]
85
86 print("Subject-wise Highest:")
87 for i in range(3):
88     print("Subject",i+1,"->",h[i])
89
90 # 13 subject-wise average
91 avg=[0,0,0]
92 for a,b,c in x:
93     for i in range(3):
94         avg[i]+=c[i]
95 avg=[z/len(x) for z in avg]
96
97 print("Subject-wise Average:")
98 for i in range(3):
99     print("Subject",i+1,"->",format(avg[i], '
100
101 # 14 above 80 in at least 2 subs
102 q=0
103 for a,b,c in x:
```



```
104     if sum(1 for y in c if y>80)>=2:
105         q+=1
106 print("Count above 80 in at least two subject")
107 print(q)
108
109 # 15 highest in subject 3
110 m3=max(x, key=lambda r:r[2][2])
111 print("Student with highest mark in Subject 3")
112 print(m3[1], "-", m3[2][2])
113
114 # 16 data structure exp (short)
115 print("Data structure explanation:")
116 print("Tuple for fixed record (id,name,mark: ")
```



Input

stdin input

Output



Run



Save